

# Recapitulação de Python e POO

UMA SEMANA DEPOIS

## vamos reconstruir um RPG funcional

Você não precisa ter nenhum código anterior.

Cada arquivo começa vazio e cada peça será testada antes da próxima.

ARQUIVO NOVO

```
$ touch retomada.py
```

ler

prever

digitar

executar

# Contrato da retomada

## HOJE VALE

- Começar com arquivo vazio
- Copiar somente o bloco atual
- Prever antes de executar
- Usar print para observar valores
- Pedir ajuda mostrando o erro

## HOJE NÃO VALE

- Depender do projeto antigo
- Colar o jogo inteiro de uma vez
- Apagar a mensagem de erro
- Mudar cinco partes antes de testar
- Responder sem acompanhar o estado

**META: sair com um programa pequeno que você consegue explicar.**

# Aquecimento 1: variáveis e saída

## PYTHON

```
nome = "Thoric"
vida = 100
ataque = 15

print("Personagem:", nome)
print("Vida:", vida)
print("Ataque:", ataque)

vida = vida - 20
print("Vida depois do dano:", vida)
```

## ANTES DE RODAR

- Qual valor muda?
- Qual valor continua igual?
- O que aparece na última linha?

## SAÍDA

```
> Personagem: Thoric
> Vida: 100
> Ataque: 15
> Vida depois do dano: 80
```

Primeiro rastreie valores simples.

## Aquecimento 2: input e tipos

### PYTHON

```
nome = input("Nome do herói: ")
vida = int(input("Vida inicial: "))

print(nome, "começa com", vida, "de vida")

dano = int(input("Dano recebido: "))
vida = vida - dano

if vida < 0:
    vida = 0

print("Vida atual:", vida)
```

`input()` > texto

`int(input())` > inteiro

### TESTE

vida 50, dano 20 > 30

vida 50, dano 80 > 0

## Aquecimento 3: escolher uma ação

### PYTHON

```
print("1 - Atacar")
print("2 - Defender")
print("3 - Usar poção")

escolha = input("Escolha: ")

if escolha == "1":
    print("Você atacou")
elif escolha == "2":
    print("Você se defendeu")
elif escolha == "3":
    print("Você usou uma poção")
else:
    print("Opção inválida")
```

### COMPARE TIPOS IGUAIS

"1" == "1" True

"1" == 1 False

### Experimente 1, 2, 3 e 9.

O else impede que uma escolha inesperada quebre o fluxo.

# Aquecimento 4: repetir até sair

## PYTHON

```
escolha = ""

while escolha != "3":
    print("1 - Jogar")
    print("2 - Ver regras")
    print("3 - Sair")
    escolha = input("Escolha: ")

    if escolha == "1":
        print("Jogo iniciado")
    elif escolha == "2":
        print("Reduza a vida do adversário")
    elif escolha != "3":
        print("Opção inválida")

print("Programa encerrado")
```

## SIGA O FLUXO

- testa a condição
- mostra o menu
- lê a escolha
- executa um caminho
- volta à condição

O laço termina porque  
**escolha muda para "3".**

# Arquivo novo: primeira classe

## PYTHON

```
class Personagem:
    def __init__(self, nome, vida, ataque):
        self.nome = nome
        self.vida = vida
        self.ataque = ataque

thoric = Personagem("Thoric", 100, 15)
goblin = Personagem("Goblin", 45, 8)

print(thoric.nome, thoric.vida)
print(goblin.nome, goblin.vida)
```

## DIGITE DO ZERO

- A classe é o único molde
- Cada chamada cria um objeto
- Cada objeto guarda seus valores
- self muda a cada criação

## SAÍDA

```
> Thoric 100
> Goblin 45
```

# Primeiro método: mostrar\_status()

## PYTHON

```
class Personagem:
    def __init__(self, nome, vida, ataque):
        self.nome = nome
        self.vida = vida
        self.ataque = ataque

    def mostrar_status(self):
        print(self.nome, "-", self.vida, "de vida")

thoric = Personagem("Thoric", 100, 15)
goblin = Personagem("Goblin", 45, 8)

thoric.mostrar_status()
goblin.mostrar_status()
```

## NA DEFINIÇÃO

```
def mostrar_status(self)
```

## NA CHAMADA

```
thoric.mostrar_status()
```

Python envia Thoric para self.



# Método que muda estado: receber\_dano()

## PYTHON

```
class Personagem:
    def __init__(self, nome, vida, ataque):
        self.nome = nome
        self.vida = vida
        self.ataque = ataque

    def receber_dano(self, quantidade):
        self.vida = self.vida - quantidade

        if self.vida < 0:
            self.vida = 0

goblin = Personagem("Goblin", 45, 8)
goblin.receber_dano(20)
print(goblin.vida)
```

## TESTE 3 CASOS

- dano 20 > vida 25
- dano 0 > vida não muda
- dano 100 > vida 0

O objeto protege  
**a própria vida.**

# Dois objetos: atacar(alvo)

## ADICIONE À CLASSE ATUAL

```
class Personagem:  
    # mantenha __init__ e receber_dano  
  
    def atacar(self, alvo):  
        print(self.nome, "ataca", alvo.nome)  
        alvo.receber_dano(self.ataque)  
  
thoric = Personagem("Thoric", 100, 15)  
goblin = Personagem("Goblin", 45, 8)  
  
thoric.atacar(goblin)  
print("Vida do Goblin:", goblin.vida)
```

`self = thoric`

origem do ataque

`alvo = goblin`

recebe o dano

**Saída esperada: Vida do Goblin: 30**

# Evolução opcional: defesa e poção

## DEFESA

```
def defender(self):  
    self.defendendo = True  
  
def receber_dano(self, quantidade):  
    if self.defendendo:  
        quantidade = quantidade // 2  
        self.defendendo = False  
  
    self.vida = max(0, self.vida - quantidade)
```

## POÇÃO

```
def usar_pocao(self):  
    if self.pocoes > 0 and self.vida < self.vida_maxima:  
        self.vida = self.vida + 25  
  
        if self.vida > self.vida_maxima:  
            self.vida = self.vida_maxima  
  
    self.pocoes = self.pocoes - 1
```

Antes de usar, acrescente pocoes ao construtor e crie:

```
vida_maxima = vida      |      pocoes = valor recebido      |      defendendo = False
```

# Aleatoriedade sem esconder a regra

## PYTHON

```
import random

def atacar(self, alvo):
    rolagem = random.randint(1, 20)
    print("d20:", rolagem)

    if rolagem < 5:
        print(self.nome, "errou")
    else:
        dano = random.randint(
            self.ataque - 3,
            self.ataque + 3
        )
        alvo.receber_dano(dano)
        print("Dano:", dano)
```

1 a 4 erro

5 a 20 acerto

### SE ERROU

não calcule dano

não chame receber\_dano

# Função de turno: conectar escolhas

## PYTHON

```
def turno_do_jogador(jogador, inimigo):  
    print("1 - Atacar")  
    print("2 - Defender")  
    print("3 - Usar poção")  
  
    escolha = input("Escolha: ")  
  
    if escolha == "1":  
        jogador.atacar(inimigo)  
    elif escolha == "2":  
        jogador.defender()  
    elif escolha == "3":  
        jogador.usar_pocao()  
    else:  
        print("Opção inválida")
```

## A FUNÇÃO

- mostra opções
- lê texto
- escolhe um caminho
- chama métodos existentes

**Não refaça o dano**

dentro do menu.

# Batalha mínima: monte o while

## PYTHON

```
jogador = Personagem("Thoric", 100, 15, 2)
inimigo = Personagem("Goblin", 45, 8, 0)

while jogador.vida > 0 and inimigo.vida > 0:
    jogador.mostrar_status()
    inimigo.mostrar_status()

    turno_do_jogador(jogador, inimigo)

    if inimigo.vida > 0:
        inimigo.atacar(jogador)

if jogador.vida > 0:
    print("Você venceu!")
else:
    print("Você foi derrotado.")
```

## ORDEM

- conferir duas vidas
- mostrar estado
- jogador age
- conferir inimigo
- inimigo age se vivo

**and: ambos precisam estar vivos.**

# Teste guiado antes de adicionar mais

## CRIAÇÃO

os dois status aparecem?

## DANO

a vida nunca fica negativa?

## ALVO

o objeto certo perde vida?

## DEFESA

vale por um ataque?

## POÇÃO

respeita limite e quantidade?

## LAÇO

termina e mostra resultado?

Se um teste falhar, pare ali. O próximo bloco depende dele.

# Depuração: cinco erros para corrigir

## NÃO EXECUTE AINDA

```
class Personagem:
    def __init__(nome, vida):          # 1
        nome = nome                   # 2
        self.vida = vida

escolha = input("1 - Atacar: ")
if escolha == 1:                      # 3
    jogador.atacar(inimigo)

while jogador.vida > 0 or inimigo.vida > 0: # 4
    turno_do_jogador(jogador, inimigo)
    inimigo.atacar(jogador)           # 5
```

## PROCURE

- self ausente
- atributo não salvo
- texto contra inteiro
- or no lugar de and
- ataque depois da derrota

Explique o efeito do erro,  
**não apenas a correção.**



# Saída da retomada

## CRIAR

```
obj = Personagem(...)
```

## LER

```
obj.mostrar_status()
```

## MUDAR

```
obj.receber_dano(10)
```

## INTERAGIR

```
jogador.atacar(inimigo)
```

## ESCOLHER

```
input() retorna texto
```

## REPETIR

```
while vida1 > 0 and  
      vida2 > 0:
```

Você começou sem arquivo anterior e terminou com uma batalha explicável.